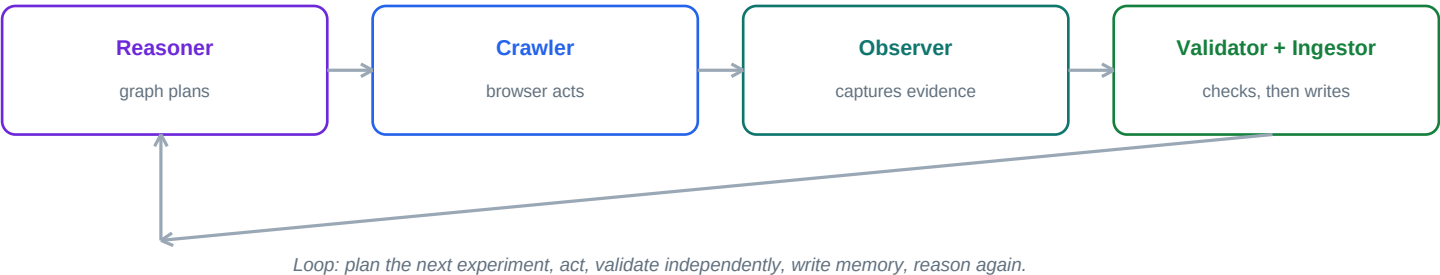


# Part B - Production Architecture

Turning the Amazon-checkout prototype into something a real testing platform could run on - the part of the stack a team leans on to decide what is actually safe to ship.

The architecture is built around two things that are painful to change later: the agent harness, where agents live, and the evaluation loop, where you prove the system is improving instead of quietly rotting.



The core loop: the graph plans, the browser acts, the harness validates and writes memory, and the graph reasons over what it learned.

## 0. The one belief this rests on

A browser agent and a knowledge graph are good at almost opposite things. The browser agent is opportunistic: it can work out which control adds an item to cart after the DOM changes, but it only exercises the paths it happens to walk and can report success on an action that quietly did nothing.

The graph is the part that remembers: every state seen, every control, what was proved versus merely noticed, and what should exist but has not been found. The crawler discovers; the graph remembers and cross-examines; the harness stops the loop from lying to itself.

## 1. The multi-agent network

The work is split into small agents with hard boundaries rather than one clever mega-agent. Each agent owns one kind of decision, and the hand-off is a typed contract rather than a shared blob of state.

| Agent             | Owns                                       | LLM?                         | Boundary enforced                                            |
|-------------------|--------------------------------------------|------------------------------|--------------------------------------------------------------|
| Planner           | Frontier, priorities, budgets              | No                           | Decides what to explore; never touches the browser.          |
| Crawler           | One UI action via browser-use / Playwright | Small / fast                 | Executes how; never writes graph truth or decides pass/fail. |
| Observer          | Snapshot: DOM, screenshot, URL, controls   | Mostly no                    | Returns raw evidence, not a trusted summary.                 |
| Reasoner          | Missed, absent, causal scenarios           | LLM proposes; Cypher decides | Deterministic graph queries remain the trustworthy floor.    |
| Validator         | Independent post-conditions                | No                           | Nothing is validated until evidence is rechecked.            |
| Healer            | Selector repair                            | Small model + graph          | Suggests fixes; Validator confirms.                          |
| Triage            | Failure clustering and flake calls         | LLM + graph history          | Escalates; does not silently suppress.                       |
| Safety supervisor | Pre-action guardrails                      | No                           | Vetoes before the browser sees the task.                     |

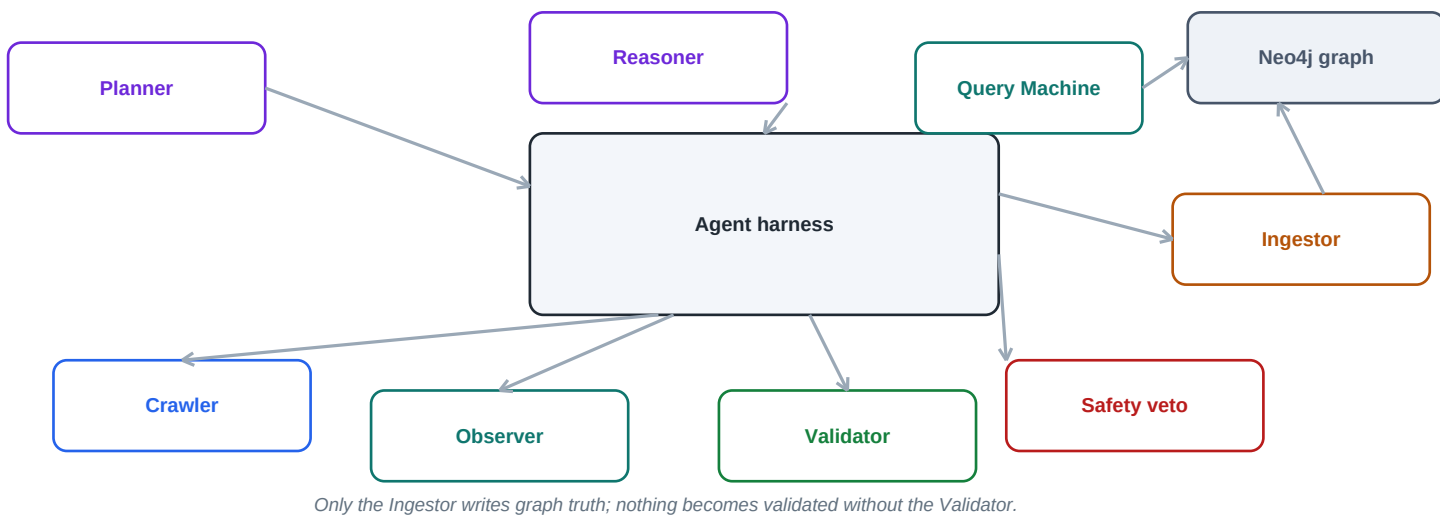
| Agent    | Owns                    | LLM? | Boundary enforced                              |
|----------|-------------------------|------|------------------------------------------------|
| Ingestor | Idempotent graph writes | No   | Typed, replayable writes only; no model calls. |

The most important boundary is between Crawler and Validator. When the Crawler says it changed the quantity, the Validator reads the cart again and checks that the number actually moved.

- **Fails:** retry with backoff up to a budget, then mark the step failed and continue. A failed step is data, not a crash.
- **Stalls:** an action ledger and no-repeat guard turn repeated clicks into a fast veto.
- **Low confidence:** escalate, hand off to a human, or write an explicit unverified marker. Uncertainty is a first-class output.

### 1.1 The contract between agents

Every hand-off carries intent, expected state, confidence, evidence to seek, risk class, and escalation signal. Those Pydantic-style contracts are the durable interface that lets models or frameworks change underneath.



## The agent harness - this is the architecture

The harness owns state, retries, timeouts, tool dispatch, schema enforcement, deterministic replay, and human-in-the-loop interrupts. LangGraph can provide state-machine primitives; the durable value sits above it.

- **State and replay:** durable Temporal workflows record inputs, outputs, model and prompt versions, trace IDs, and artifacts.
- **Structured dispatch:** agents return schema-validated objects; malformed output is a caught failure.
- **Retries and budgets:** latency, token, and circuit-breaker policies are observable, not silent spend.
- **HITL interrupts:** runs pause cleanly, ask a human, then resume with provenance intact.

## 3. Guardrails built into each agent

One shared safety layer across the whole system is the wrong shape because each agent is unsure about different things. Guardrails attach to each agent individually: every agent declares what not confident enough means, and what happens when it crosses that line.

**The Part-A deny-list safety veto is the right shape: default-allow reversible exploration, hard-block enumerable irreversible actions such as purchase, payment, sign-out, and navigation away from the product under test.**

| Guardrail                              | What it does                                                                                |
|----------------------------------------|---------------------------------------------------------------------------------------------|
| Output validators / schema enforcement | The output must fit the contract or the step fails.                                         |
| Confidence gating                      | Below threshold: escalate, fall back, or write unverified. No silent low-confidence passes. |
| Circuit breakers                       | Repeated failures trip a feature-level stop and surface a flag.                             |
| Fallback chains                        | Frontier model to small model to deterministic rule to human, per agent.                    |
| Deny-list veto                         | Irreversible actions blocked before execution by a separate process.                        |

## 4. Model routing and composition

The job is choosing the right model for each task and combining them without one model's guess turning into the next one's fact. The principle is deterministic by default, small models for high-volume fuzzy work, and frontier models only where reasoning value is high and volume is low.

| Task                                               | Route              | Why                                                                 |
|----------------------------------------------------|--------------------|---------------------------------------------------------------------|
| State resolution, validation, inference, ingestion | Deterministic code | About 90 percent of operations; must be explainable and repeatable. |
| Which element to click                             | Small / fast model | High volume, latency-sensitive UI skill.                            |

| Task                                                      | Route                  | Why                                   |
|-----------------------------------------------------------|------------------------|---------------------------------------|
| Locator stability / flake classification                  | Fine-tuned small model | Narrow, repetitive, cheaper at scale. |
| Graph gap reasoning, root-cause chains, eval adjudication | Frontier model         | Low volume, high reasoning value.     |

- Chain models without letting errors snowball: put deterministic checks or graph lookups between model stages.
- Pin prompt and model versions so provider changes are measured before customers file a ticket.
- Keep long sessions small by using the graph as long-term memory and querying only the needed slice.
- Keep frontier models off the hot path; cache small-model calls by graph-state signature and invalidate bounded subgraphs.

## 5. The production graph schema

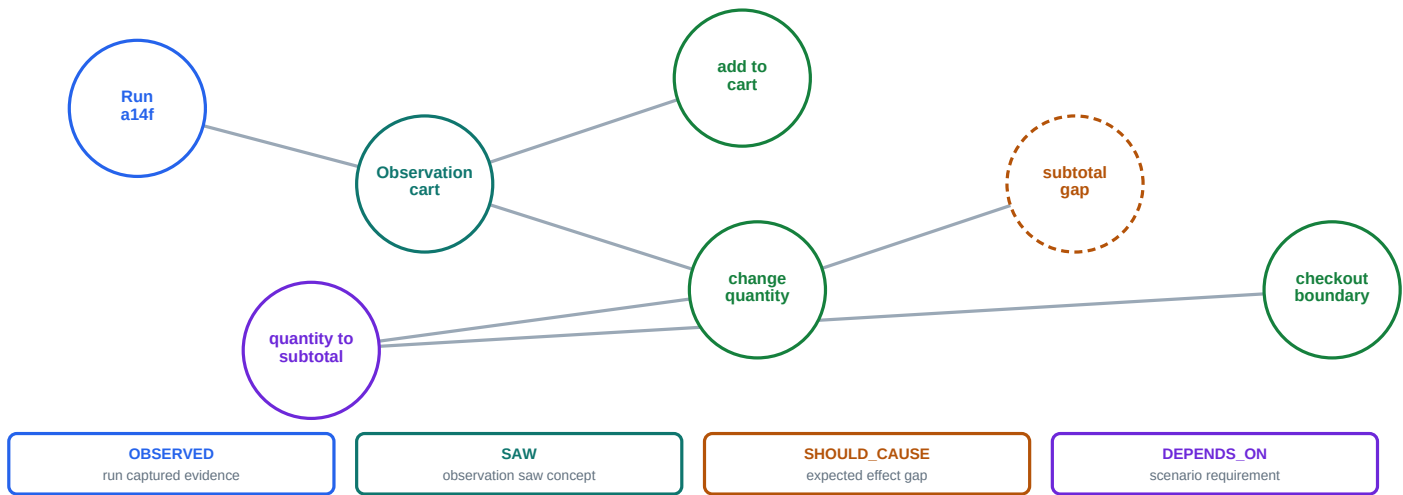
The graph is not built around Element to Component to Flow to Feature. It is built around provenance and absence: why we believe something, and what should exist but does not.

| Node                   | Key properties                                       | Question answered                                    |
|------------------------|------------------------------------------------------|------------------------------------------------------|
| Tenant / App / Feature | ids                                                  | Scope one feature deeply without tenant bleed.       |
| Run                    | commit_sha, started_at, model_version, cost          | What did we know at run N versus N+1?                |
| Observation            | state, url, artifact_ref, screenshot_ref             | Replay exactly what the browser saw.                 |
| Concept                | key, kind, observed, validated, expected, confidence | Expected-not-observed; observed-not-validated.       |
| Intent                 | source, status, risk, selector_ref, confidence       | Why an action ran, passed, or was vetoed.            |
| Scenario               | key, status, confidence, last_confirmed_run          | Which scenarios are trusted now versus decayed?      |
| Selector               | hash, role/css/xpath, stability, last_failed         | Blast radius and self-healing.                       |
| CodeArtifact           | path, symbol, commit_sha                             | Map PR change to selectors to concepts to scenarios. |

**The unit of meaning is a Concept - a behaviour - not a DOM element. Selectors churn; the meaning of 'add to cart' survives redesigns.**

The edges are where most of the value lives, so provenance and confidence belong on edges too: where the relationship came from, and how sure the system is of it.

| Edge                   | Between                       | What it lets us ask                                       |
|------------------------|-------------------------------|-----------------------------------------------------------|
| OBSERVED / SAW_CONCEPT | Run to Observation to Concept | What did we see, and where?                               |
| TARGETS                | Intent to Concept             | Which attempt exercised which behaviour, and did it pass? |
| DEPENDS_ON             | Scenario to Concept           | Which concepts does this scenario rely on?                |
| SHOULD_CAUSE           | Concept to Concept            | Which expected effect remains unproven?                   |
| BACKS                  | CodeArtifact to Selector      | Which code backs this locator?                            |



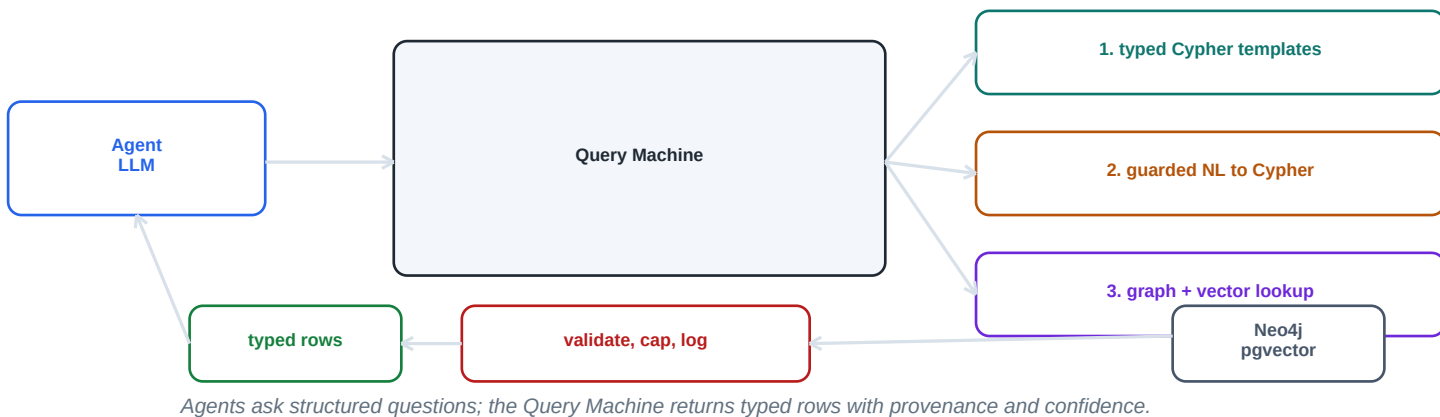
A sample checkout graph: the dashed subtotal node and SHOULD\_CAUSE edge are the causal gap the reasoner surfaces.

## 5.1 Why a graph and not just vectors

- **Absence:** missing tests are structural gaps, not similarity queries.
- **Temporal reasoning:** knowledge at commit X needs stamped edges and history.
- **Multi-hop traversal:** CodeArtifact to Selector to Concept to Scenario is the product.
- Vectors still help with selector repair and semantic matching, but only as an auxiliary index.

## 6. The Query Machine

The Query Machine lives between agents and Neo4j. It turns 'what does the graph know?' into bounded, typed answers instead of dumping the graph into a prompt or letting an agent write arbitrary Cypher.



- **Tier 1:** reviewed, parameterised graph-query tools such as `missed_scenarios`, `absence`, `blast_radius`, `neighbours`, `confidence_of`, and `what_did_we_know_at`.
- **Tier 2:** guarded natural-language to Cypher for the long tail: read-only, schema allow-listed, row/cost capped, logged, and reviewable.
- **Tier 3:** hybrid graph plus vector lookup: similarity finds the starting node, structure performs the reasoning.

## 7. Keeping the graph correct over time

| Operation   | Trigger                                                             | How                                                                                                                    |
|-------------|---------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Incremental | Every crawl                                                         | Upsert observations, bump <code>last_seen</code> , append evidence, add <code>Scenario-[CONFIRMED_BY]-&gt;Run</code> . |
| Recomputed  | A run touching a feature                                            | Re-run inference and confidence for that feature only; never global.                                                   |
| Invalidated | PR touches mapped code; selector fails repeatedly; validation fails | Mark a bounded subgraph stale and push affected scenarios into the retest frontier.                                    |
| Compounds   | Many runs over time                                                 | Selector stability, flake rate, transition probabilities, and confidence-decay curves.                                 |

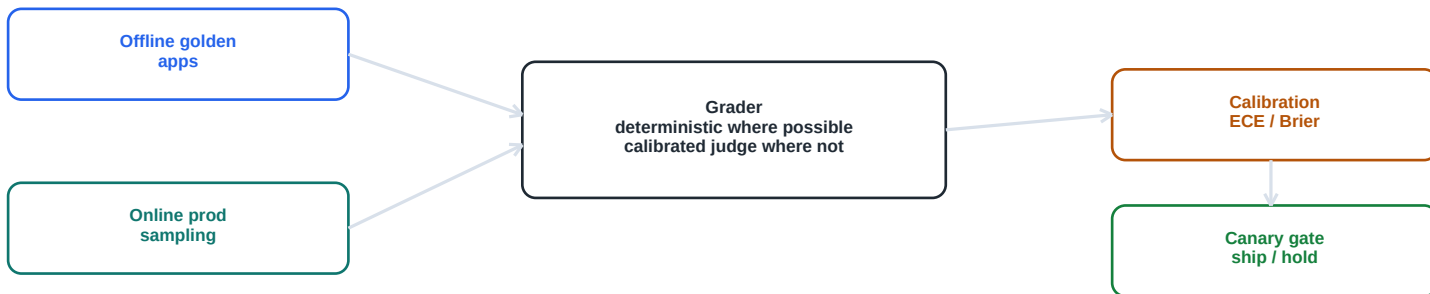
- Conflict resolution preserves history instead of overwriting: old and new observations remain stamped by commit and time, so what the system believed at commit X stays answerable.
- Validated decisions feed the next run: confirmed scenarios raise priors; repeatedly flaky ones lower them.
- Full nightly recompute is rejected as the freshness story. Bounded invalidation is the defensible production shape.



Confidence decays when runs do not reconfirm a scenario and snaps back on proof. Below the retest line it re-enters the frontier.

## 8. Eval and confidence

This is the layer that decides whether the system is a product or a science project. It protects against two confidently-wrong states: the agent clicked the wrong thing, and the graph inferred a scenario that is not real.



Offline golden sets and online sampling feed graders; calibration and canary gates stand between a model change and customers.

| Question                       | Signal                                            | Metric                         |
|--------------------------------|---------------------------------------------------|--------------------------------|
| Did it click the right target? | Trace plus positive/negative target check         | Wrong-target rate, wander rate |
| Did the action succeed?        | Deterministic post-condition after re-observe     | Validation pass rate           |
| Is the scenario reproducible?  | Replay across clean sessions                      | Flake rate, pass@k             |
| Are confidence scores honest?  | Predicted confidence versus observed reproduction | ECE, Brier score               |
| Did a model change regress us? | Canary on versioned golden trajectories           | Delta versus baseline          |

Use a managed eval store to move fast, but own the golden sets and grading logic in-house. LLM-as-judge must be calibrated against human spot-checks, and never used where a deterministic check exists.

## 9. Observability and operations

- One trace id follows the whole chain of agent calls, with model and prompt versions stamped at each hop.
- Decision audit trails record frontier score, guardrail choice, confidence, and veto/pass outcome.
- Latency and cost budgets per stage make stalls visible.
- Alerts focus on agentic failure modes: wander-rate drift, validation spikes, stale concepts, and model-upgrade regressions.
- Rollout moves from observe-only to recommend to gated execution to autonomous safe execution after thresholds clear.

## 10. Multi-tenancy and scale

| Layer            | Isolated per tenant                                                  | Shared                                      |
|------------------|----------------------------------------------------------------------|---------------------------------------------|
| Graph            | Tenant/app/feature partition; database-per-tenant for large accounts | Schema and inference engine code            |
| Artifacts        | Per-tenant namespace, retention, redaction                           | Storage implementation                      |
| Models / prompts | Customer vocabulary and trace versions                               | Generic templates                           |
| Learning         | No selectors, DOM, screenshots, or scenarios cross tenants           | Aggregated, reviewed structural priors only |

Scale shape: async agents on a queue, durable Temporal workflows, LLM-call caching by graph-state signature, bounded Query Machine traversals, and heavy DOM/screenshot artifacts in object storage while Neo4j stores references.

## 11. PR blast radius

The PR hook falls straight out of the graph: CodeArtifact to Selector to Concept to Scenario to confidence. The output should say exactly which selectors, validated scenarios, and inferred scenarios are at risk - not 'rerun all checkout tests.'

## 12. What I would refuse to ship

| Refusal                                   | Why, and what must exist first                                                                      |
|-------------------------------------------|-----------------------------------------------------------------------------------------------------|
| A path that can click final purchase      | Money pages must be structurally incapable of final submit; red-team it until it cannot buy.        |
| LLM self-report as validation             | Every mutating action needs a deterministic post-condition.                                         |
| Uncalibrated confidence in the UI         | Confidence without calibration is false authority.                                                  |
| Nightly full-graph recompute as freshness | Expensive and hides drift; prove bounded invalidation on a golden tenant.                           |
| Unreviewed cross-tenant learning          | Only reviewed structural priors may cross tenants; never DOM, selectors, screenshots, or scenarios. |
| Hard-coded concept vocabulary as-is       | Fine for one feature; production needs a learned per-tenant concept model behind an adapter.        |

**The hardest problem is knowing the system is wrong before a human notices. The architecture answers with deterministic floors under the fuzzy parts, confidence that is measured rather than assumed, canaries between model changes and customers, and a graph that remembers what used to be true so drift is visible.**

## 13. How Part A grounds every claim

| Production claim                                   | Prototype seed                                                          |
|----------------------------------------------------|-------------------------------------------------------------------------|
| Planner / Crawler split; graph plans, browser acts | GraphGuidedExplorer._autonomous_loop; BrowserUseIntentExecutor          |
| Guardrails an LLM cannot bypass                    | safety_guard.veto_reason in the browser-use callback                    |
| Executor output is evidence, not truth             | re-observe plus _run_assertions; cart-delta / checkout verification     |
| Graph reasons about missed and causal gaps         | infer_missed_scenarios; CAUSAL_EXPECTATIONS / SHOULD_CAUSE              |
| Query Machine tier 1 and NL taste                  | infer_missed_scenarios; blast_radius; graph_expansion.expand_from_graph |
| Absence as graph query                             | seed_expected_concepts; missing_expected_concepts                       |
| PR blast radius traversal                          | pr_blast_radius.py; GraphStore.blast_radius                             |
| Provenance and confidence on writes                | write_intent; write_observation                                         |

## 14. The stack, and why

| Layer             | Start with                                                                                         | Avoid                                                  |
|-------------------|----------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| Agent harness     | Custom harness over LangGraph primitives; Temporal for durable runs                                | A fully autonomous agent with no state/replay contract |
| Browser execution | Playwright plus browser-use, sandboxed                                                             | No action contract or veto                             |
| Graph store       | Neo4j with temporal provenance and confidence on edges                                             | Graph-shaped dependencies hidden in relational tables  |
| Retrieval         | pgvector / Weaviate as auxiliary hybrid lookup                                                     | Vector similarity replacing graph truth                |
| Query Machine     | Parameterised Cypher tools plus guarded NL-to-Cypher                                               | Free-form model-written Cypher on the hot path         |
| Models            | Frontier reasoning/judge; small executors; fine-tuned narrow classifiers; deterministic validation | Provider name as the whole strategy                    |
| Eval              | Versioned golden apps, managed eval store, calibrated judge, canaries                              | Shipping prompt changes without canaries               |



| Layer         | Start with                                   | Avoid                                     |
|---------------|----------------------------------------------|-------------------------------------------|
| Observability | OpenTelemetry, trace IDs, decision audit     | Plain logs with no trace IDs              |
| Async / queue | Celery + Redis, FastAPI, Docker/K8s, AWS/GCP | Fire-and-forget cron for retryable crawls |
| Artifacts     | S3/GCS; Neo4j holds refs only                | Large DOM/screenshot blobs inside Neo4j   |

**In one line: the valuable part is not the clever crawler. It is the loop between something that explores and something that remembers, with a hard wall between what the system did and what it is allowed to believe.**